

Software Requirements Specification: EasyService

Version: 1.0

Status: Pilot-oriented academic specification

System: EasyService Marketplace Platform

1. Purpose and scope

EasyService enables customers to discover local services, review availability and price, create bookings, pay using a simulated wallet, and manage booking status. Providers can publish listings and manage available quantity. The system is intentionally small enough for formal testing and demonstrates a complete transaction workflow.

2. Actors

- **Customer:** searches listings, books, pays, views and cancels transactions.
- **Provider:** creates and manages listings and inventory.
- **Administrator:** manages platform configuration and operational oversight.
- **External payment service:** represented by a test double in the academic implementation.
- **Identity service:** represented by a deterministic verification double.

3. Functional requirements

ID	Requirement	Acceptance evidence
FR-01	The system shall display listings by category and location.	Filtered catalogue result
FR-02	The system shall validate customer identity type and identity value.	Verified or rejected registration
FR-03	The system shall prevent duplicate email and phone registration.	Registration service tests
FR-04	The system shall allow booking only for published listings.	Booking guard test
FR-05	The system shall reject non-positive quantity and price values.	BVA tests
FR-06	The system shall reject a quantity greater than current availability.	Capacity test
FR-07	The system shall calculate promotions deterministically.	Promotion service tests
FR-08	The system shall create a pending transaction before payment resolution.	State transition evidence

FR-09	Successful payment shall confirm the transaction and reduce inventory.	Unit, integration, and system tests
FR-10	Failed payment shall expire the transaction without deducting funds.	Payment failure test
FR-11	The system shall support cancellation and restore eligible inventory and balance.	Cancellation test
FR-12	The system shall reject forbidden state transitions.	Negative state tests
FR-13	The UI shall expose a multi-step booking journey.	Selenium system scenarios
FR-14	CI shall run automated tests on repository changes.	GitHub Actions and Jenkins

4. Non-functional requirements

- **NFR-01 Reliability:** booking and inventory mutations must be atomic within the selected persistence design.
- **NFR-02 Security:** production deployment must use real authentication, secret management, authorization, TLS, and audit logs.
- **NFR-03 Performance:** pilot listing search should respond within 2 seconds at the agreed pilot load.
- **NFR-04 Accessibility:** interactive controls must be keyboard reachable and have meaningful labels.
- **NFR-05 Maintainability:** business rules remain in service classes with unit-level tests.
- **NFR-06 Observability:** production must record correlation IDs, transaction outcomes, and reconciliation failures without logging sensitive data.
- **NFR-07 Testability:** the system shall support deterministic test doubles for identity, payment, notification, and randomness.

5. Business rules

- BR-01 Ethiopian customers use Fayda identity; foreign customers use passport identity.
- BR-02 Only verified customers may book.
- BR-03 Listing price and capacity must be positive.
- BR-04 Listing must be published to accept a booking.
- BR-05 Requested quantity must be at least one and no greater than available quantity.
- BR-06 Payment succeeds only when the wallet can cover the final total.
- BR-07 Valid promotions reduce the final total according to their rule.
- BR-08 Confirmed bookings reduce inventory exactly once.
- BR-09 Eligible cancellation restores funds and inventory exactly once.
- BR-10 Completed, expired, and cancelled transactions cannot be confirmed again.

6. State model

PENDING -> CONFIRMED -> COMPLETED

PENDING -> EXPIRED on payment failure

PENDING -> CANCELLED on cancellation

CONFIRMED -> CANCELLED on eligible cancellation

Forbidden transitions must raise a domain error and leave balance and inventory unchanged.

7. Data requirements

Core entities are **User**, **Listing**, **Transaction**, and **Promotion**. The academic implementation uses in-memory repositories. Production requires a transactional relational store, migration scripts, retention rules, backup, recovery, and audit history.

8. Interfaces

- Svelte/Vite browser frontend.
- Spring Boot REST API.
- Simulated payment and identity interfaces for tests.
- CI interfaces through Maven, npm, GitHub Actions, and Jenkins.

9. Acceptance and release constraints

Acceptance requires passing unit and integration suites, executed Selenium evidence, no open critical defects, documented residual risk, and a verified coverage report. Commercial release additionally requires production persistence, real payment compliance, privacy review, security testing, and operational support readiness.